# 0242. Valid Anagram

asy | Category: Arrays & Hashing | LeetCode: https://leetcode.com/problems/vali

# Problem

- Given strings s and t, return true if t is a rearrangement of the letters of s.
- Example: s = "anagram", t = "nagaram" → true; s = "rat", t = "car" → false.

# Intuition

- Two strings are anagrams exactly when every character occurs the same number of times in both.
- With only lowercase letters, a 26-slot count array is a perfect, tiny hash map.

# Brute Force

- Sort both strings and compare: O(n log n) time.

# Optimal Approach

- If lengths differ, return false immediately.
- For each index i: count[s[i]]++ and count[t[i]]--.
- If every count is zero at the end, the multisets match.

# Worked Example

```
s = "rat", t = "car".

i=0  +r -c   r:1 c:-1
i=1  +a -a   a:0
i=2  +t -r   t:1 r:0
counts not all zero (c=-1, t=1) -> false
```

# Java Solution

```java
public class ValidAnagram {
    public boolean isAnagram(String s, String t) {
        if (s.length() != t.length()) return false;
        int[] count = new int[26];
        for (int i = 0; i < s.length(); i++) {
            count[s.charAt(i) - 'a']++;
            count[t.charAt(i) - 'a']--;
        }
        for (int c : count) if (c != 0) return false;
        return true;
    }
}
```

# Complexity

- Time: O(n).
- Space: O(1), the array always has 26 slots. For Unicode use a HashMap (O(k) distinct chars).

# Pitfalls

- Forgetting the length check makes "a" vs "ab" look valid if you only scan s.
- For the Unicode follow-up, iterate code points, not chars.